

Hackathon 2.0

Knowvia

Software Product Requirements Document (PRD)

Department-Scoped Scheduling, Materials, Assignments, Chat & Announcements

September 2026.

1. Document Information

Field	Value
Product Name	Knowvia
Solution Type	Software - Progressive Web Application (PWA)
Prepared By	The Algorithm
Document Version	1.0
Date	10 th September 2026

Product Summary

Knowvia is a department-scoped progressive web app built for a multi-department tech hub internship program. It replaces informal channels such as WhatsApp and Telegram with a single, professional workspace for class scheduling, learning materials, assignment management, announcements, and group chat. Every department operates in its own fully isolated space, and every uploaded file is validated and screened before it is stored.

2. Problem Statement

Interns currently receive class updates, learning materials, and announcements through informal chat apps, where they are easily buried or lost, and the hub still relies on a static, inflexible timetable. There is no shared, department-isolated space for this content, and files shared informally carry no safety checks.

Problem Evidence / Assumptions to Validate

- Interns need a single, reliable place to check their class schedule and catch new announcements.
- A static timetable cannot accommodate ad-hoc classes that tutors want to add.
- Materials and assignment submissions need type/size validation and malware screening before they are shared.
- Each department's schedule, materials, chat, and announcements must remain fully isolated from every other department; only the admin sees across all of them.

3. Product Goal & Objectives

Product Goal

To give every department a single, isolated, professional workspace covering class scheduling, learning materials, assignment management, announcements, and group chat — replacing the hub's static timetable and its reliance on informal chat apps.

Project Objectives

ID	Objective	How Completion Will Be Gauged
01	A tutor/admin creates or edits a class schedule entry; it becomes the primary dashboard view for both tutors and students.	Entry created in Department A is invisible in Department B; appears as the top dashboard item for both roles.
02	A tutor uploads a learning material of any file type	Disallowed/oversized file rejected;

	within the size limit; it is validated and scanned before storage.	malicious test file blocked; valid file uploads and is visible to students.
03	A tutor creates an assignment, which auto-generates an entry on the student's announcement bell linking to full detail.	Clicking the bell entry takes the student to the correct assignment record.
04	A student submits completed assignment work; the dashboard progress meter reflects the milestone.	Submission is stored and the progress meter updates accordingly.

4. Target Users & Personas

User	Need	Typical Action	Success Condition
Intern	Schedule-first dashboard, safe materials, assignment tracking, chat	View schedule, download materials, submit assignments, chat	Everything relevant is in one department-scoped place
Tutor	Reach only their department; share materials/assignments safely	Post schedule/materials/assignments, review submissions, chat	Manage the department without cross-department noise
Admin	Oversight across all departments	Create departments, post hub-wide announcements	Full visibility; others stay scoped to their own department

Primary Persona Example

SaWizz, 24, is a cybersecurity intern. He wants his dashboard to open straight to this week's schedule, grab a lab file his tutor uploaded, see what his current assignment is and how far along he is, and get notified the moment his tutor adds an extra class or posts an announcement, all without cross-department noise.

5. Value Proposition & Significance

- Interns: a schedule-first dashboard, safely screened materials, and visible assignment progress.
- Tutors: one professional, safe channel to reach their department and manage assignments.
- The hub: files are screened before storage, and departments stay fully isolated from one another.

6. Proposed Solution & User Experience

The MVP is a responsive, installable PWA with a clean, light UI (soft, minimal colors, clear spacing) that adapts naturally across screen sizes. Both tutor and student dashboards lead with the class schedule, since it replaces the hub's static timetable. Materials, assignments, chat, and the announcement bell are reachable from there, all scoped to the user's department.

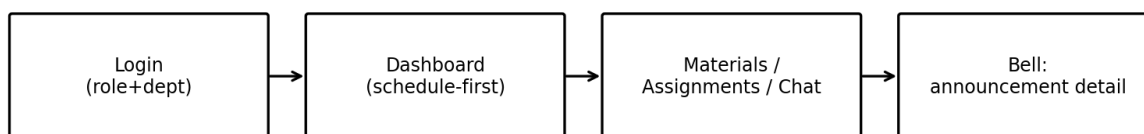


Figure 1. Primary user flow.

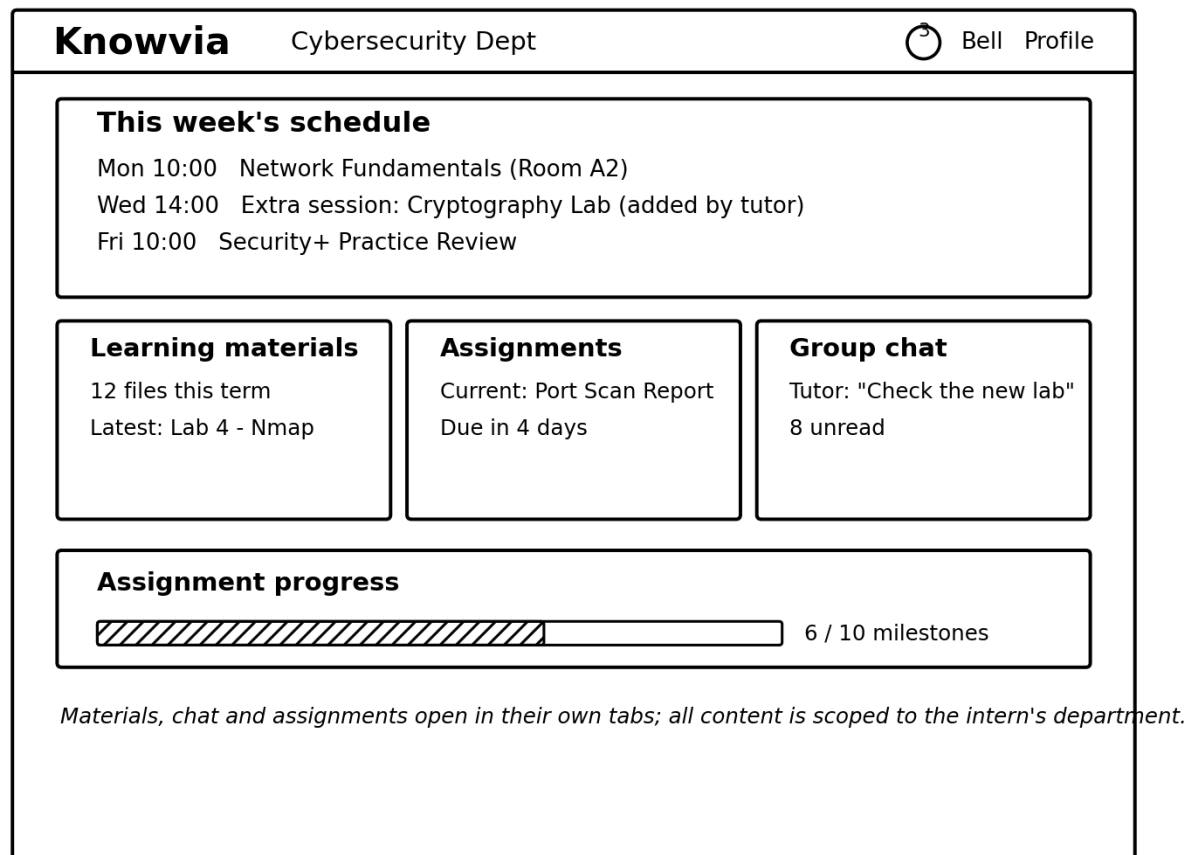


Figure 2. Low-fidelity wireframe of the student dashboard (schedule-first).

7. Functional Requirements

ID	Requirement	Priorit y	User	Acceptance Summary
FR-01	Authenticate and scope session to role + department	Must	All	User only ever sees their own department's data
FR-02	Tutor/admin creates, edits, deletes class schedule entries	Must	Tutor/Admin	Entry appears only within that department
FR-03	Schedule shown as the primary dashboard view for all roles	Must	All	Schedule is the first thing shown after login
FR-04	Tutor uploads materials (any type, size-limited)	Must	Tutor	File stored and linked to uploader's department
FR-05	Uploads validated for type/size and scanned for malware before storage	Must	System	Malicious/oversized file never reaches storage
FR-06	Students view/download department materials	Must	Intern	Only same-department materials are listed
FR-07	Tutor creates an assignment; auto-generates an announcement entry	Must	Tutor	Announcement links to the new assignment
FR-08	Announcement bell links to full assignment/material/schedule detail	Must	Intern	Bell click opens the correct record

FR-09	Student submits completed assignment work (any type, size-limited)	Must	Intern	Submission stored and linked to the assignment
FR-10	Progress meter and current-assignment view on student dashboard	Must	Intern	Meter reflects completed milestones accurately
FR-11	Department-scoped chat; shows sender name; reply-to-message	Must	All	Only same-department members see/send messages
FR-12	Admin announcements are hub-wide; tutor announcements are department-only	Must	Admin/Tutor	Scope matches the poster's role
FR-13	Chat attachments pass the same validation/scan as materials	Could	Admin/Tutor System	Unsafe chat attachment is rejected
FR-14	Push notification on new announcement; reminder ~1 day before a scheduled class	Should	All	User is alerted without manually refreshing

8. Non-Functional Requirements

Area	Requirement	Target / Constraint	Verification
Security	No cross-department data access, including chat	No endpoint returns another department's rows	Access-control tests
File safety	Type/size validation and malware scan on every upload	Unsafe files never reach storage	Positive/negative upload tests
Usability	Schedule-first flow stays simple on mobile	3 taps or fewer after login	Task walkthrough
Performance	Pages and chat respond promptly under demo load	Most requests under 2 seconds in test	Basic timing tests
Cost	Runs entirely on free-tier hosting	No paid service required for the MVP	Deployment review
Privacy	Collect only information required for the MVP	No unnecessary personal data stored	PRD/data review

9. MVP Scope & Prioritization

The MVP proves the full loop: schedule-first dashboards, safe materials, assignment tracking with a progress meter, announcements, and chat — all fully department-isolated.

Priority	Included	Hackathon Decision
Must	Auth + department scoping, schedule (dashboard-first), materials with validation/scan, assignments + submissions + progress meter, announcements (bell + tab), chat with the same file checks	Build and test first (Week 2)
Should	Push notifications, class-reminder notifications	Build once Must features are stable (Week 3)
Could	Chat read receipts / presence	Only if time remains

Out of Scope	Native mobile apps, video calls, multi-hub tenancy	Document for future work
--------------	--	--------------------------

10. System Architecture & Technical Design

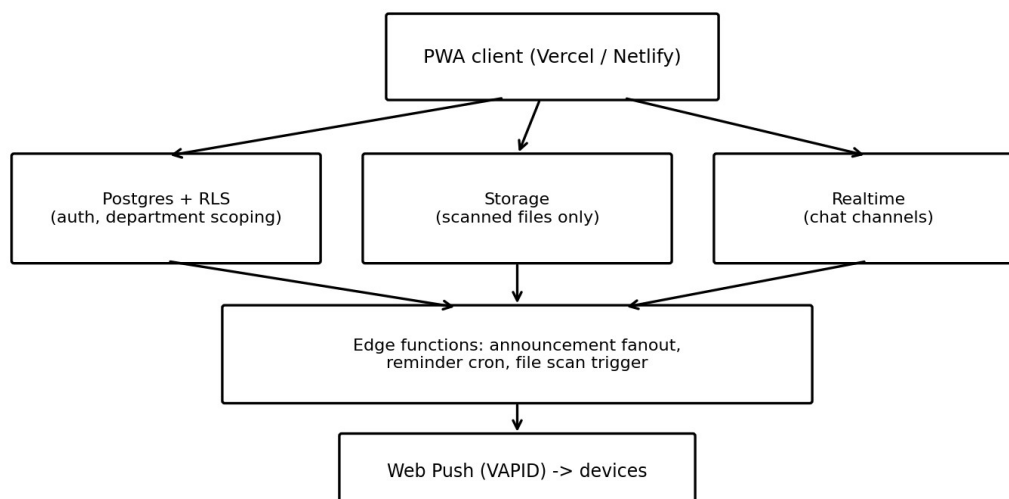


Figure 3. Illustrative architecture — fully free-tier (Vercel/Netlify + Supabase).

Proposed Technology Stack

Layer	Illustrative Choice	Rationale
Frontend	Vite + React 19 + TypeScript	Installable, supports push, adapts across screen sizes
Backend	Express.js + TypeScript	Mature, stable API environment.
Database	Supabase PostgreSQL (Postgres, Auth, Storage, Realtime, Edge Functions)	RLS enforces department segregation at the DB layer, including chat
File scanning	Type/extension allowlist + malware scan before storage	Blocks unsafe files before they reach storage or chat
Scheduling	Supabase pg_cron for reminder job	Free, built into the same platform, no extra service needed
Deployment	Vercel or Netlify (frontend), Supabase (backend) — both free tier	Zero hosting cost for a hackathon MVP

Core Data Entities

Department, User, ClassSchedule, Resource (materials), Assignment, Submission, Announcement (generic — links back to its source via ref_type/ref_id), Conversation, Message, PushSubscription.

Key Technical Decisions

- Department segregation enforced at the database layer (RLS), covering schedule, materials, assignments, and chat alike.

- Creating a schedule entry, material, or assignment auto-generates an announcement row; edge functions fan out notifications by department (or hub-wide for admin).
- File validation (type/size/malware scan) is one shared step used by both material uploads and chat attachments.

11. Acceptance Criteria & Success Metrics

Objective	Acceptance Criterion	Evidence	Pass Condition
01 - Scheduling	Schedule is department-scoped and dashboard-first	Live demo, two departments	Cross-department schedule not visible; shows on login
02 - File safety	Unsafe files rejected; valid files upload/download	Positive/negative upload tests	Malicious/disallowed file never reaches storage
03 - Assignments	Bell links to full detail; progress meter updates	Live demo	Click-through and milestone view both work correctly
04 - Chat	Chat and attachments are department-scoped and validated	Live demo + attachment test	Cross-department user blocked; unsafe attachment rejected

12. Risks, Constraints & Assumptions

Risk / Constraint	Impact	Likelihood	Mitigation
Free malware scanning is limited (library setup or rate-limited API)	Medium	Medium	Start with a strict type/size allowlist; add scanning once core flow works
Segregation bug leaks data or chat across departments	High	Low-Medium	Enforce via RLS in addition to API checks; test access control early
Limited 3-week build time against a broad feature set	High	Medium	Freeze Must-have scope early (section 9)

Assumptions

- The MVP is a hackathon proof of concept.
- Departments and roles are pre-seeded for the demo with the option of self-service registration.

13. Development Plan & Milestones

Period	Primary Outcome	Exit Check
Week 1 - Planning	This PRD, data model, architecture, repository setup	Scope and architecture agreed
Week 2 - MVP	Auth, schedule, materials, assignments, chat, announcements working end to end	Two departments demonstrate full segregation and the core loop
Week 3 - Final Product	Push/reminders added, bug fixes, rehearsed	Reliable demo and clear scope

	demo	explanation
--	------	-------------

14. Open Questions / Decisions Pending

Question	Current Decision / Direction
What counts as a disallowed file type?	Block executables/scripts; allow common document/image/media types for MVP
Malware scanning: library (ClamAV) or third-party API?	To be decided during a Week 1 technical spike
One chat channel per department, or more?	One shared channel per department for MVP